TRƯỜNG ĐẠI HỌC KỸ THUẬT CÔNG NGHIỆP

KHOA ĐIỆN TỬ

BÀI TIỂU LUẬN

KỸ THUẬT LẬP TRÌNH NÂNG CAO

NGÀNH : KỸ THUẬT MÁY TÍNH

HỆ : ĐẠI HỌC CHÍNH QUY

ĐỀ TÀI: XÂY DỰNG PHẦN MỀM GIẢI SUDOKU BẰNG QUAY LUI

THÁI NGUYÊN - 2026

TRƯỜNG ĐẠI HỌC KỸ THUẬT CÔNG NGHIỆP

KHOA ĐIỆN TỬ

BÀI TIỂU LUẬN

KỸ THUẬT LẬP TRÌNH NÂNG CAO

BỘ MÔN : CÔNG NGHỆ THÔNG TIN

ĐỀ TÀI: XÂY DỰNG PHẦN MỀM GIẢI SUDOKU BẰNG QUAY LUI

GIÁO VIÊN HƯỚNG DẪN	:	TS. NGHIÊM VĂN TÍNH
HỌ VÀ TÊN SINH VIÊN	:	NGUYỄN NHƯ KHIÊM
		ĐẬU VĂN KHÁNH
LỚP	:	K58KTP
MSSV	:	K225480106030
		K225480106099

THÁI NGUYÊN - 2026

TRƯỜNG ĐHKTCN CỘNG HOÀ XÃ HỘI CHỦ NGHĨA VIỆT NAM

KHOA ĐIỆN TỬ **Độc lập - Tự do - Hạnh phúc**

PHIẾU GIAO BÀI TIỂU LUẬN KỸ THUẬT LẬP TRÌNH NÂNG CAO

Sinh viên: Nguyễn Như Khiêm MSSV: K225480106030

Đậu Văn Khánh MSSV: K225480106099

Lớp: K58KTP Khoá: 2022-2027

Ngành học: Kỹ thuật máy tính

Giáo viên hướng dẫn: TS. Nghiêm Văn Tính

1. Tên đề tài: Xây dựng phần mềm giải Sudoku bằng quay lui

2. Nội dung thực hiện:

- Yêu cầu về lý thuyết: Trình bày cơ sở lý thuyết liên quan, phân tích bài toán, thuật toán sử dụng, đánh giá độ phức tạp.
- Yêu cầu về thực nghiệm/ kết quả: Xây dựng chương trình hoàn chỉnh bằng một ngôn ngữ cụ thể (C/C++, Java hoặc Python), có giao diện người dùng thân thiện; chạy thử nghiệm với dữ liệu mẫu và trình bày kết quả.

3. Các sản phẩm, kết quả:

- Thuyết minh báo cáo.
- Demo phầm mềm

4. Ngày giao nhiệm vụ: 24/12/2025

5. Ngày hoàn thành nhiệm vụ: 10/03/2026

TRƯỞNG KHOA	**BỘ MÔN**	**GIÁO VIÊN HƯỚNG DẪN**
(Ký và ghi rõ họ tên)	*(Ký và ghi rõ họ tên)*	*(Ký và ghi rõ họ tên)*

NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Thái Nguyên, ngày….tháng…..năm 2026

GIÁO VIÊN HƯỚNG DẪN

(Ký ghi rõ họ tên)

MỤC LỤC

LỜI CẢM ƠN 5
LỜI NÓI ĐẦU 6
CHƯƠNG 1: TỔNG QUAN VỀ MÔN HỌC VÀ ĐỀ TÀI 7
1.1. Tổng quan về môn học kỹ thuật lập trình nâng cao 7
1.2. Tổng quan về đề tài 8
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT 11
2.1. Thuật toán tổng quát 11
2.2. Công nghệ áp dụng 13
CHƯƠNG 3: PHƯƠNG PHÁP QUAY LUI 15
3.1. Khái niệm quay lui 15
3.2. Mô hình của bài toán 16
3.3. Ứng dụng 17
CHƯƠNG 4: BÀI TOÁN GIẢI SUDOKU 20
4.1. Giới thiệu bài toán 20
4.2. Thiết kế tối ưu thuật toán 21
4.3. Giới thiệu chương trình giải 25
CHƯƠNG 5: CÀI ĐẶT CHƯƠNG TRÌNH THỬ NGHIỆM 28
5.1. Chạy chương trình 28
5.2. Chạy chương trình kết thúc 29
CHƯƠNG 6: ĐÁNH GIÁ VÀ HƯỚNG PHÁT TRIỂN 31
6.1. Những kết quả đạt được 31
6.2. Những kết quả chưa đạt được 31
6.3. Hướng phát triển 31
KẾT LUẬN 32
TÀI LIỆU THAM KHẢO 33

LỜI CẢM ƠN

Nhóm em xin bày tỏ lòng biết ơn sâu sắc đến thầy **Nghiêm Văn Tính**, giảng viên Trường Đại học Kỹ thuật Công nghiệp Thái Nguyên, người đã tận tình hướng dẫn, chỉ bảo và đồng hành cùng nhóm em trong suốt quá trình thực hiện và hoàn thiện báo cáo này. Nhờ sự giúp đỡ quý báu của thầy, nhóm em đã có thể vượt qua những khó khăn, từng bước hoàn thành tốt nội dung đề tài được giao.

Nhóm em cũng xin gửi lời cảm ơn chân thành tới các thầy giáo, cô giáo trong Khoa Điện tử và các thầy, cô thuộc Bộ môn Công nghệ Thông tin – Trường Đại học Kỹ thuật Công nghiệp Thái Nguyên, những người đã luôn tận tâm giảng dạy, truyền đạt kiến thức, tạo nền tảng vững chắc cho em trong suốt thời gian học tập tại trường.

Mặc dù đã cố gắng hết sức, nhưng trong quá trình thực hiện báo cáo, chắc chắn sẽ không tránh khỏi những thiếu sót nhất định. Nhóm em rất mong nhận được sự thông cảm và những góp ý quý báu từ các thầy, cô để nhóm em có thể rút kinh nghiệm, hoàn thiện bản thân hơn trong những chặng đường học tập và công tác sau này.

Em xin chân thành cảm ơn!

LỜI NÓI ĐẦU

Trong bối cảnh khoa học – công nghệ không ngừng phát triển, đặc biệt trong các lĩnh vực như Điện tử, Công nghệ thông tin và Viễn thông, con người ngày càng phải nâng cao năng lực bản thân để theo kịp xu thế hiện đại. Sự phát triển của đất nước gắn liền với tiến bộ khoa học kỹ thuật, đòi hỏi mỗi cá nhân không chỉ có kiến thức chuyên môn mà còn cần tư duy logic, khả năng phân tích và giải quyết vấn đề hiệu quả.

Một trong những phương pháp rèn luyện tư duy hiệu quả là thông qua các trò chơi trí tuệ, đặc biệt là những trò chơi liên quan đến con số. Sudoku là một trò chơi trí tuệ phổ biến, có luật chơi đơn giản nhưng đòi hỏi người chơi phải suy luận logic và cẩn thận để tìm ra lời giải. Nhờ tính linh hoạt và phù hợp với nhiều lứa tuổi, Sudoku trở thành công cụ hữu ích giúp rèn luyện tư duy và phát triển trí não.

Trong khi đó, thuật toán quay lui là một thuật toán điển hình để giải lớp bài toán liệt kê hay bài toán tối ưu như: bài toán người giao hàng, bài toán con mã đi tuần, bài toán 8 hậu, bài toán tìm đường đi trong mê cung... Bằng việc liệt kê các tình huống, thử các khả năng có thể cho đến khi tìm thấy một lời giải đúng, thuật toán quay lui chia nhỏ bài toán, lời giải của bài toán lớn sẽ là kết quả của việc tìm kiếm theo chiều sâu của tập hợp các bài toán phần tử.

Nhận thấy tư tưởng của thuật toán rất phù hợp với cách giải của trò chơi, nên đề tài "Xây dựng phần mềm giải Sudoku bằng quay lui" được thực hiện nhằm tìm hiểu về thuật toán quay lui và ứng dụng của nó.

CHƯƠNG 1: TỔNG QUAN VỀ MÔN HỌC VÀ ĐỀ TÀI

1.1. Tổng quan về môn học kỹ thuật lập trình nâng cao

1.1.1. Giới thiệu chung về môn học:

Trong bối cảnh công nghệ thông tin và khoa học máy tính phát triển mạnh mẽ, lập trình ngày càng đóng vai trò quan trọng trong hầu hết các lĩnh vực kỹ thuật và đời sống. Không chỉ dừng lại ở việc viết các chương trình đơn giản, người học cần nắm vững những kỹ thuật lập trình nâng cao để giải quyết các bài toán phức tạp, có yêu cầu cao về hiệu năng và tính tối ưu. Môn học Kỹ thuật lập trình nâng cao được xây dựng nhằm đáp ứng nhu cầu đó, giúp sinh viên nâng cao tư duy thuật toán và năng lực lập trình.

1.1.2. Mục tiêu của môn học:

Môn học Kỹ thuật lập trình nâng cao hướng tới các mục tiêu chính sau:

- Trang bị cho sinh viên kiến thức nền tảng về các kỹ thuật và phương pháp thiết kế thuật toán.
- Rèn luyện tư duy logic, tư duy thuật toán và khả năng phân tích, giải quyết vấn đề.
- Giúp sinh viên biết cách lựa chọn và áp dụng thuật toán phù hợp cho từng bài toán cụ thể.
- Nâng cao khả năng lập trình, tối ưu chương trình và đánh giá hiệu quả của thuật toán.

1.1.3. Nội dung chính của môn học:

Nội dung môn học tập trung vào các kỹ thuật lập trình và thuật toán quan trọng, bao gồm:

- Phân tích bài toán và mô hình hóa bài toán bằng thuật toán.
- Phân tích độ phức tạp thuật toán về thời gian và bộ nhớ.
- Các kỹ thuật thiết kế thuật toán phổ biến như:
  + Chia để trị (Divide and Conquer).
  + Tham lam (Greedy).
  + Quy hoạch động (Dynamic Programming).
  + Quay lui (Backtracking).
  + Nhánh và cận (Branch and Bound).

- Ứng dụng các kỹ thuật trên để giải các bài toán kinh điển trong tin học như đệ quy, tìm kiếm, sắp xếp, tối ưu hóa và các bài toán tổ hợp.

1.1.4. Phương pháp học tập và thực hành:

Bên cạnh kiến thức lý thuyết, môn học đặc biệt chú trọng đến thực hành lập trình. Sinh viên được yêu cầu:

- Cài đặt các thuật toán bằng ngôn ngữ lập trình cụ thể (C/C++, Java, Python,…).
- Thực hiện các bài tiểu luận, bài tập tình huống và đồ án môn học.
- Rèn luyện kỹ năng viết mã rõ ràng, tối ưu và dễ bảo trì.
- Thông qua đó, sinh viên không chỉ hiểu thuật toán về mặt lý thuyết mà còn biết cách áp dụng hiệu quả vào thực tế.

1.1.5. Vai trò và ý nghĩa của môn học:

Môn Kỹ thuật lập trình nâng cao có vai trò quan trọng trong chương trình đào tạo ngành kỹ thuật và công nghệ thông tin. Đây là môn học:

- Làm nền tảng cho các môn học chuyên ngành như trí tuệ nhân tạo, học máy, xử lý ảnh, phân tích dữ liệu và phát triển phần mềm.
- Giúp sinh viên hình thành tư duy giải quyết vấn đề một cách khoa học và có hệ thống.
- Góp phần nâng cao chất lượng nguồn nhân lực, đáp ứng yêu cầu ngày càng cao của thị trường lao động trong thời đại số.

1.2. Tổng quan về đề tài

1.2.1. Lý do chọn đề tài:

Trong quá trình học tập các môn liên quan đến lập trình, cấu trúc dữ liệu và giải thuật, việc áp dụng lý thuyết vào các bài toán thực tế có ý nghĩa quan trọng trong việc nâng cao tư duy logic và kỹ năng lập trình của sinh viên. Sudoku là một trò chơi trí tuệ quen thuộc, có luật chơi đơn giản nhưng lại chứa đựng nhiều ràng buộc logic chặt chẽ, rất phù hợp để nghiên cứu và áp dụng các thuật toán tìm kiếm.

Thuật toán quay lui (Backtracking) là một trong những thuật toán cơ bản nhưng hiệu quả trong việc giải các bài toán có không gian tìm kiếm lớn và nhiều ràng buộc. Việc áp dụng thuật toán quay lui để giải bài toán Sudoku không chỉ giúp người học hiểu rõ hơn nguyên lý hoạt động của thuật toán mà còn rèn luyện khả năng phân tích, kiểm tra điều kiện và xử lý các trường hợp sai.

Bên cạnh đó, việc xây dựng phần mềm giải Sudoku có giao diện trực quan giúp người dùng dễ dàng tương tác, quan sát quá trình giải bài toán, từ đó tăng tính trực quan và sinh động trong học tập. Chính vì những lý do trên, đề tài "Xây dựng phần mềm giải Sudoku bằng thuật toán quay lui" được lựa chọn để nghiên cứu và thực hiện.

1.2.2. Mục tiêu của đề tài:

Mục tiêu chính của đề tài là nghiên cứu và xây dựng một phần mềm có khả năng giải bài toán Sudoku dựa trên thuật toán quay lui. Cụ thể, đề tài hướng đến các mục tiêu sau:

- Nghiên cứu nguyên lý hoạt động của thuật toán quay lui (Backtracking).
- Phân tích bài toán Sudoku và các ràng buộc cần thỏa mãn.
- Xây dựng chương trình giải Sudoku tự động bằng ngôn ngữ lập trình Python.
- Thiết kế giao diện đồ họa giúp người dùng nhập dữ liệu, quan sát quá trình giải và xem kết quả.
- Kiểm tra và đánh giá tính đúng đắn, hiệu quả của chương trình.

1.2.3. Phạm vi và đối tượng nghiên cứu:

❖ Phạm vi nghiên cứu:

- Bài toán Sudoku kích thước chuẩn 9×9.
- Áp dụng thuật toán quay lui để tìm lời giải.
- Sử dụng ngôn ngữ lập trình Python và thư viện Pygame để xây dựng giao diện.
- Không nghiên cứu các thuật toán tối ưu nâng cao hoặc Sudoku kích thước lớn hơn.

❖ Đối tượng nghiên cứu:

- Thuật toán quay lui (Backtracking).
- Bài toán Sudoku và các quy tắc ràng buộc.
- Quy trình xây dựng phần mềm giải bài toán logic.
- Người dùng là sinh viên hoặc người học lập trình có nhu cầu tìm hiểu thuật toán.

1.2.4. Phương pháp nghiên cứu:

- Tìm hiểu thông tin trên mạng Internet, sách, báo, tạp chí...

- Thu thập ý kiến chuyên gia (giáo viên hướng dẫn, ý kiến bạn bè...).

- Kết hợp nghiên cứu lý thuyết với thực hành, rèn luyện kỹ năng phân tích chương trình và kỹ năng lập trình.

1.2.5. Ý nghĩa khoa học và thực tiễn:

❖ Ý nghĩa khoa học:

- Giúp củng cố và vận dụng kiến thức về thuật toán quay lui.

- Minh họa rõ ràng cách áp dụng thuật toán vào bài toán có ràng buộc.

- Là tài liệu tham khảo cho sinh viên trong quá trình học tập các môn giải thuật và lập trình.

❖ Ý nghĩa thực tiễn:

- Phần mềm hỗ trợ giải Sudoku nhanh chóng và chính xác.

- Giúp người học quan sát trực quan quá trình giải bài toán.

- Có thể mở rộng để nghiên cứu các thuật toán khác hoặc nâng cấp chức năng trong tương lai.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1. Thuật toán tổng quát

2.1.1. Mảng

❖ Khái niệm và các đặc trưng cơ bản:

- Khái niệm: Mảng là một dãy có thứ tự (về mặt vị trí) các phần tử với 2 đặc điểm:

  + Số phần tử cố định.

  + Mọi phần tử đều có cùng kiểu dữ liệu.

- Các đặc trưng cơ bản:

  + Mảng cho phép truy cập ngẫu nhiên đến từng phần tử. Thời gian truy cập đến mọi phần tử đều bằng nhau.

  + Số lượng phần tử trong mảng là cố định.

❖ Cấu trúc lưu trữ:

- Cấu trúc đơn giản nhất của mảng là dùng địa chỉ được tính để thực hiện lưu trữ và tìm kiếm các phần tử.

- Các phần tử được bố trí sát nhau trong bộ nhớ và theo thứ tự tăng dần của các chỉ số nên ta có thể dễ dàng tìm được địa chỉ của một phần tử bất kì nếu biết chỉ số của nó.

a_0	a_1	a_2	…	…	a_i	…	…	a_{n-1}

❖ Các phép toán cơ bản:

- Thường chỉ có các phép tạo lập (create) mảng, tìm kiếm (retrieve) một phần tử của mảng, cập nhật (update) một phần tử của mảng... Ngoài giá trị, một phần tử của mảng còn được đặc trưng bởi chỉ số (index) thể hiện thứ tự của phần tử đó trong mảng. Vector là mảng một chiều, mỗi phần tử a_i của nó ứng với một chỉ số i. Ma trận là mảng hai chiều, mỗi phần tử a_{ij} ứng với hai chỉ số i và j. Tương tự, người ta cũng mở rộng ra mảng hai chiều, mảng ba chiều,..., mảng n chiều.

2.1.2. Hàm đệ quy:

❖ Khái niệm:

- Một đối tượng là đệ quy nếu nó bao gồm chính nó như một bộ phận hoặc nó được định nghĩa dưới dạng của chính nó.

- Một thủ tục được gọi là đệ quy nếu trong quá trình thực hiện nó phải gọi đến chính nó nhưng kích thước nhỏ hơn của tham số.

- Trong thân một hàm, nếu nó gọi tới chính hàm đó để xử lý bài toán thì gọi là hàm đệ quy.

❖ Cấu trúc chính của một chương trình đệ quy:

Một chương trình đệ quy gồm 2 phần:

- Phần cơ sở: Chứa các tác động của hàm hoặc thủ tục với một số giá trị cụ thể ban đầu của tham số (còn được gọi là trường hợp dừng).
- Phần đệ quy: Trong đó, tác động cần được thực hiện cho giá trị hiện thời của các tham số được định nghĩa bằng các tác động đã được định nghĩ trước đấy với kích thước nhỏ hơn của tham số.

❖ Đặc điểm:

- Mỗi một lần hàm tự gọi đệ quy đến nó thì máy tính sẽ tạo ra một biến cục bộ mới.

- Có bao nhiêu lần hàm gọi đệ quy thì sẽ có bấy nhiêu lần thoát ra khỏi hàm (kiểu như lặp hàm).

- Khi thoát ra ngoài hàm đệ quy thì một loạt các biến cục bộ tạo ra do dùng đệ quy lúc này được giải phóng và chúng sẽ giải phóng trước các biến cục bộ (sinh ra do đệ quy) tạo ra sau.

- Sử dụng đệ quy là một phương pháp làm cho chương trình ngắn gọn, dễ hiểu nhưng nó sẽ làm tốn bộ nhớ và thời gian.

❖ Ưu điểm và nhược điểm của hàm đệ quy:

- Ưu điểm:

+ Điểm mạnh lớn nhất: Chương trình, code trở nên ngắn gọn, dễ hiểu, thuận lợi cho việc chỉnh sửa.

+ Dễ chuyển thành chương trình trên các ngôn ngữ lập trình.

- Nhược điểm:

+ Nhược điểm lớn nhất là tốn bộ nhớ.

+ Mất nhiều thời gian xử lý, làm giảm tốc độ chạy chương trình.

+ Không áp dụng cho mọi ngôn ngữ lập trình.

- Đệ quy không những là một phương pháp lập trình quan trọng mà còn là một phương pháp suy nghĩ để giải quyết vấn đề một cách tổng quát dựa trên ý tưởng:

+ Đơn giản hóa công việc.

+ Phân vùng để xử lý.

2.2. Công nghệ áp dụng

2.2.1. Ngôn ngữ lập trình Python:

❖ Khái niệm: Python là một ngôn ngữ lập trình bậc cao cho các mục đích lập trình đa năng do Guido van Rossum tạo ra và lần đầu ra mắt vào năm 1991. Python được thiết kế với ưu điểm mạnh là dễ đọc, dễ học và dễ nhớ.

– **Một số tính năng của Python:**

- Dễ dàng cho người mới bắt đầu: Python là ngôn ngữ có hình thức sáng sủa, cấu trúc rõ ràng, thuận tiện cho người mới học lập trình và là ngôn ngữ lập trình được sử dụng rộng rãi trong phát triển trí tuệ nhân tạo. Cấu trúc của Python còn cho phép người sử dụng viết mã lệnh với số lần gõ phím tối thiểu.

- Mã nguồn mở miễn phí: Python là ngôn ngữ lập trình khi đưa vào sử dụng có khả năng cho phép người dùng sử dụng miễn phí hoàn toàn với mọi ứng dụng, chương trình được phát triển. Với bản chất là mã nguồn mở sở hữu cộng đồng đông đảo mang lại khả năng hỗ trợ nhanh chóng, chuyên nghiệp và đầy đủ, đồng thời cũng cải tiến và cập nhật liên tục mang lại hiệu quả ứng dụng cao hơn.

- Khả năng tương thích nhiều nền tảng: Ban đầu Python được phát triển để chạy trên nền Unix. Nhưng dần theo thời gian, Python dần mở rộng sang mọi hệ điều hành từ MS – DOS đến Mac OS, Windows, Linux và các hệ điều hành khác thuộc họ Unix.

- Có thư viện lớn: Số lượng thư viện khổng lồ mà python sở hữu có khả năng đáp ứng hiệu quả, hoàn toàn được mọi nhu cầu lập trình khác nhau.thông qua đó việc

tiến hành lập trình theo yêu cầu, theo đòi hỏi thực tế của công việc là điều mà mỗi người có thể yên tâm.

- Ngôn ngữ lập trình hướng đối tượng: Đơn giản hóa đối với người mới tuy nhiên ngôn ngữ Python khi đưa vào sử dụng vẫn hoàn toàn giữ được bản chất hướng đối tượng của mình. Việc giải quyết mọi vấn đề trong lập trình được đảm bảo tốt một cách dễ hiểu và trực quan nhất.

- Python luôn được xếp hạng vào những ngôn ngữ lập trình phổ biến nhất.

2.2.2. Thư viện Pygame:

Thư viện pygame là một module open-source dành cho ngôn ngữ lập trình Python nhằm mục đích đặc biệt giúp bạn tạo trò chơi và các ứng dụng đa phương tiện khác. Được xây dựng dựa trên thư viện phát triển SDL (Simple DirectMedia Layer) có tính di động cao, pygame có thể chạy trên nhiều nền tảng và hệ điều hành.

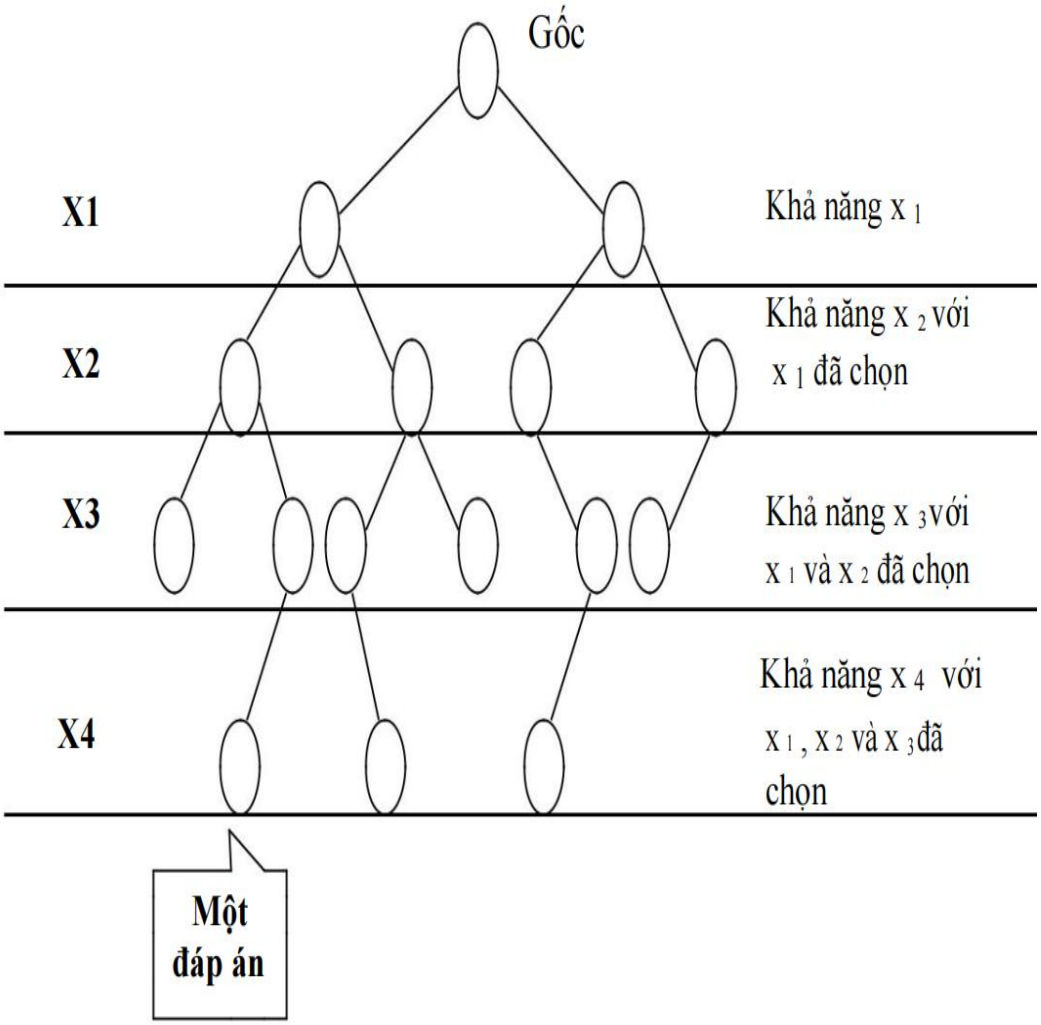
Bằng cách sử dụng module pygame, bạn có thể kiểm soát logic và đồ họa của trò chơi của bạn mà không cần lo lắng về những phức tạp backend cần thiết để làm việc với video và âm thanh.

CHƯƠNG 3: PHƯƠNG PHÁP QUAY LUI

3.1. Khái niệm quay lui

Kĩ thuật quay lui (backtracking) như tên gọi của nó, là một quá trình phân tích đi xuống và quay lui trở lại theo con đường đã đi qua. Bằng việc liệt kê các tình huống, thử các khả năng có thể có cho đến khi tìm thấy một lời giải đúng, thuật toán quay lui chia nhỏ bài toán, lời giải của bài toán lớn sẽ là kết quả của việc tìm kiếm theo chiều sâu của các bài toán phần tử. Trong suốt quá trình tìm kiếm nếu gặp phải một hướng nào đó mà biết chắc không thể tìm thấy đáp án thì quay lại bước trước đó và tìm hướng khác kế tiếp hướng vừa tìm đó. Trong trường hợp không còn một hướng nào khác nữa thì thuật toán kết thúc.

Thuật toán quay lui có thể được thể hiện theo sơ đồ cây tìm kiếm theo chiều sâu như hình dưới:



Từ hình vẽ, dễ dàng nhận thấy:

- Ở một bài toán hiện tại (mỗi nốt), ta đi tìm lời giải cho bài toán đó. Ứng với lời giải, ta đi giải bài toán kế tiếp cho đến khi bài toán gốc trở nên đầy đủ.
- Lời giải của bài toán gốc thường là một lối đi từ gốc đến nốt cuối cùng (không có nốt con).

* **Tư tưởng của thuật toán:**

- Nét đặc trưng của kĩ thuật quay lui là các bước hướng tới lời giải cuối cùng của bài toán hoàn toàn được làm thử.
- Tại mỗi bước, nếu có một lựa chọn được chấp nhận thì ghi nhận lại lựa chọn này và tiến hành các bước thử tiếp theo. Còn ngược lại không có lựa chọn nào thích hợp thì làm lại bước trước, xóa bỏ sự ghi nhận và quay về chu trình thử các lựa chọn còn lại.
- Điểm quan trọng của thuật toán là phải ghi nhớ tại mỗi bước đi qua để tránh trùng lặp khi quay lui. Các thông tin này cần được lưu trữ vào một ngăn xếp-Stack (vào sau ra trước), nên thuật toán thể hiện ý thiết kế một cách đệ quy.
- Thuật toán quay lui thường được cài đặt theo lối đệ quy, mỗi lần gọi hàm đệ quy, hàm đệ quy được truyền một tham số (trong các tham số) là chỉ số của bài toán con, trong hàm sẽ cố gắng tìm lời giải cho bài toán con đó, nếu tìm thấy thì gọi hàm đệ quy để giải bài toán con tiếp theo hoặc là đưa ra đáp án bài toán lớn nếu đã đầy đủ lời giải, nếu không tìm thấy thì chương trình sẽ trở về điểm gọi hàm đó. Mục đích của việc sử dụng hàm đệ quy là để thuật toán được rỏ ràng, dễ viết, dễ hiểu hơn và cũng để bảo toàn các biến, các trạng thái lúc giải bài toán con.

3.2. Mô hình của bài toán

Lời giải của bài toán thường biểu diễn bằng một véc tơ gồm n thành phần phải thỏa mãn các điều kiện nào đó. Để chỉ ra lời giải x, ta phải xây dựng dần các thành phần lời giải x=(x1,x2,....xn).

Tại mỗi bước i:

- Đã xây dựng xong các thành phần x=(x1,x2,....xn).

- Xây dựng thành phần xi bằng cách lần lượt thử tất cả các khả năng mà xi có thể chọn:
  + Nếu một khả năng j nào đó phù hợp cho xi thì xác định xi theo khả năng j. Thường phải có thêm thao tác ghi nhận trạng thái mới của bài toán để hỗ trợ cho bước quay lui. Nếu i = n thì ta có được một lời giải, ngược lại thì tiến hành bước i+1 để xác định xi+1 .
  + Nếu không có một khả năng nào chấp nhận được cho xi thì ta lùi lại bước trước (bước i-1) để xác định lại thành phần xi-1.

3.3. Ứng dụng

Sử dụng thuật toán quay lui dùng để giải bài toán liệt kê các cấu hình. Mỗi cấu hình được xây dựng bằng cách xây dựng từng phần tử, mỗi phần tử được chọn bằng cách thử tất cả các khả năng.

3.3.1. Phương pháp:

Giả thiết cấu hình cần liệt kê có dạng (x1 x2 ... xn). Khi đó thuật toán quay lui được thực hiện qua các bước sau :

- *Bước 1:* Xét tất cả các giá trị x1 có thể nhận, thử cho x1 nhận lần lượt các giá trị đó. Với mỗi giá trị thử cho x1 ta sẽ làm tiếp *Bước 2.*
- *Bước 2:* Xét tất cả các giá trị x2 có thể nhận, lại thử cho x2 nhận lần lượt các giá trị đó. Với mỗi giá trị thử gán cho x2 lại xét tiếp các khả năng chọn x3 ... cứ tiếp tục như vậy đến bước n.
- *Bước n:* Xét tất cả các giá trị xn có thể nhận, thử cho xn nhận lần lượt các giá trị đó, thông báo cấu hình tìm được (x1 x2 ... xn).

3.3.2. Giải thuật tổng quát :

Thuật toán quay lui có thể được mô tả bằng đoạn mã sau (thủ tục này thử cho *xi* nhận lần lượt các giá trị mà nó có thể nhận):

```
void Try (int i)
  {
    for <mọi giá trị v có thể gán cho x[i]>
      {
        <Thử cho x[i] bằng giá trị v>
        if <x[i] là phần tử cuối cùng trong cấu hình hoặc i=n>
```

```
            <Thông báo cấu hình tìm được>
        else
            {
                <Ghi nhận việc cho x[i] nhận giá trị v (nếu cần thiết)>
                Try(i+1); // gọi đệ quy chọn tiếp x[i+1]
        <Nếu cần thì bỏ ghi nhận việc thử x[i]=v để thử giá trị khác>
                }
            }
    }
```

(Thuật toán quay lui sẽ bắt đầu bằng lời gọi **Try(1)**)

+ Đầu tiên, Try(int i) là hàm đệ quy quay lui trong đó i là biến chạy từ 1 đến n tương ứng với số thứ tự của x1, x2, x3,..., xn theo phương pháp đệ quy.

+ Tiếp theo vòng lặp for cho các giá trị của từng phần tử x1, x2, x3,... Tất nhiên, đối với mỗi bài toán riêng thì những giá trị này là khác nhau.

+ Trạng thái cuối tức là điểm dừng của thuật toán này. Thực chất thuật toán quay lui cũng là đệ quy nên việc xác định điểm dừng cực kỳ quan trọng và luôn luôn phải có.

+ Tiếp theo là phần xuất ra, vì như đã thấy, khi if (phần tử cuối cùng trong cấu hình) thì đã xuất ra màn hình và không chạy bất kỳ hàm đệ quy nào nữa. Tuy nhiên, khi xuất ra nó lại xuất tới nhiều dãy số (tùy từng bài tập). Lý do, nếu để ý thì chúng ta sẽ thấy phía trên có 1 vòng for, nó sẽ hoạt động giống như kiểu nhiều vòng lặp for lồng nhau.

```
for(i =1; i<=n; i++)
for(j=1; i<n; i++)
.....
for(z=1; z<n; z++)
xuat()
```

Với m vòng for lồng nhau như vậy sẽ xuất ra n^m lần.

Như vậy nếu không đặt điều kiện nào thì vòng for đầu tiên sẽ là m lần, tiếp theo trong mỗi lần đó lại có m lần for khác.

Tóm lại, trong thủ tục mô tả trên, điều quan trọng nhất là đưa ra được một danh sách các khả năng đề cử và xác định được giá trị của biểu thức logic [chấp nhận x[i]]. Dễ thấy rằng bài toán vô nghiệm khi ta đã duyệt hết mọi khả năng mà không có khả năng nào thoả mãn yêu cầu. Chú ý rằng là đến một lúc nào đó thuật toán phải lùi liên tiếp nhiều lần. Từ đó suy ra rằng, thông thường bài toán vô nghiệm khi không thể lùi được nữa.

CHƯƠNG 4: BÀI TOÁN GIẢI SUDOKU

4.1. Giới thiệu bài toán

4.1.1. Lịch sử ra đời và luật chơi:

Theo các nguồn tư liệu, lịch sử ra đời của **sudoku** hơi phức tạp. Trò chơi này được kiến trúc sư Howard Garns ở New York thiết kế và được giới thiệu lần đầu tiên vào năm 1979 trên tạp chí Dell (Mỹ) với tên gọi là "Number Place". Tháng 4.1984, Number Place lần đầu tiên được giới thiệu tại Nhật trên báo Monthly Nikolist với tên gọi "Suuji wa dokusinh ni kagiru", dịch sang tiếng Anh có nghĩa là "những con số phải độc nhất" hoặc "những con số tìm thấy chỉ một lần" và được "rút gọn" thành sudoku (su = number, doku = single).

Luật chơi của Sudoku khá là đơn giản đó là phải điền kín những ô còn lại với điều kiện:

1. **Các hàng ngang**: Phải có đủ các số từ 1 đến 9, không cần đúng thứ tự. Không trùng số nào ở hàng ngang.
2. **Các hàng dọc**: Đảm bảo có đủ các số từ 1-9, không cần theo thứ tự. Không trùng số nào ở hàng dọc.
3. **Mỗi vùng 3 x 3**: cũng phải có đủ các số từ 1-9. Không trùng số nào trong cùng 1 vùng 3 x3.

4.1.2. Phát biểu bài toán:

Đầu vào : Cho một ma trận 9x9 và một số các số gợi ý ngẫu nhiên.

Yêu cầu : Hãy tìm ra các chữ số còn lại sao cho thỏa mãn yêu cầu.

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

5	3	4	6	7	8	9	1	2
6	7	2	1	9	5	3	4	8
1	9	8	3	4	2	5	6	7
8	5	9	7	6	1	4	2	3
4	2	6	8	5	3	7	9	1
7	1	3	9	2	4	8	5	6
9	6	1	5	3	7	2	8	4
2	8	7	4	1	9	6	3	5
3	4	5	2	8	6	1	7	9

4.2. Thiết kế tối ưu thuật toán

Phát thảo ý tưởng của bài toán như sau:

```
Try(i) ≡
        for ( j = 1 → k)
        If ( xi  chấp nhận khả năng  k)
        {
        Xác định xi theo khả năng k;
        Ghi nhận trạng thái mới
        If(i<n^2)
                Try(i+1);
        Else
                Ghi nhận nghiệm
    Trả lại trạng thái cũ cho bài toán }
```

► Vấn đề cần giải quyết cho bài toán:

- *Biểu diến sudoku như thế nào?*
- *Các khả năng cho lựa chọn xi ?*
- *Chọn xi như thế nào cho hiệu quả?*
- *Cách ghi nhận trạng thái mới? trả về trạng thái cũ ?*
- *Ghi nhận nghiệm ?*

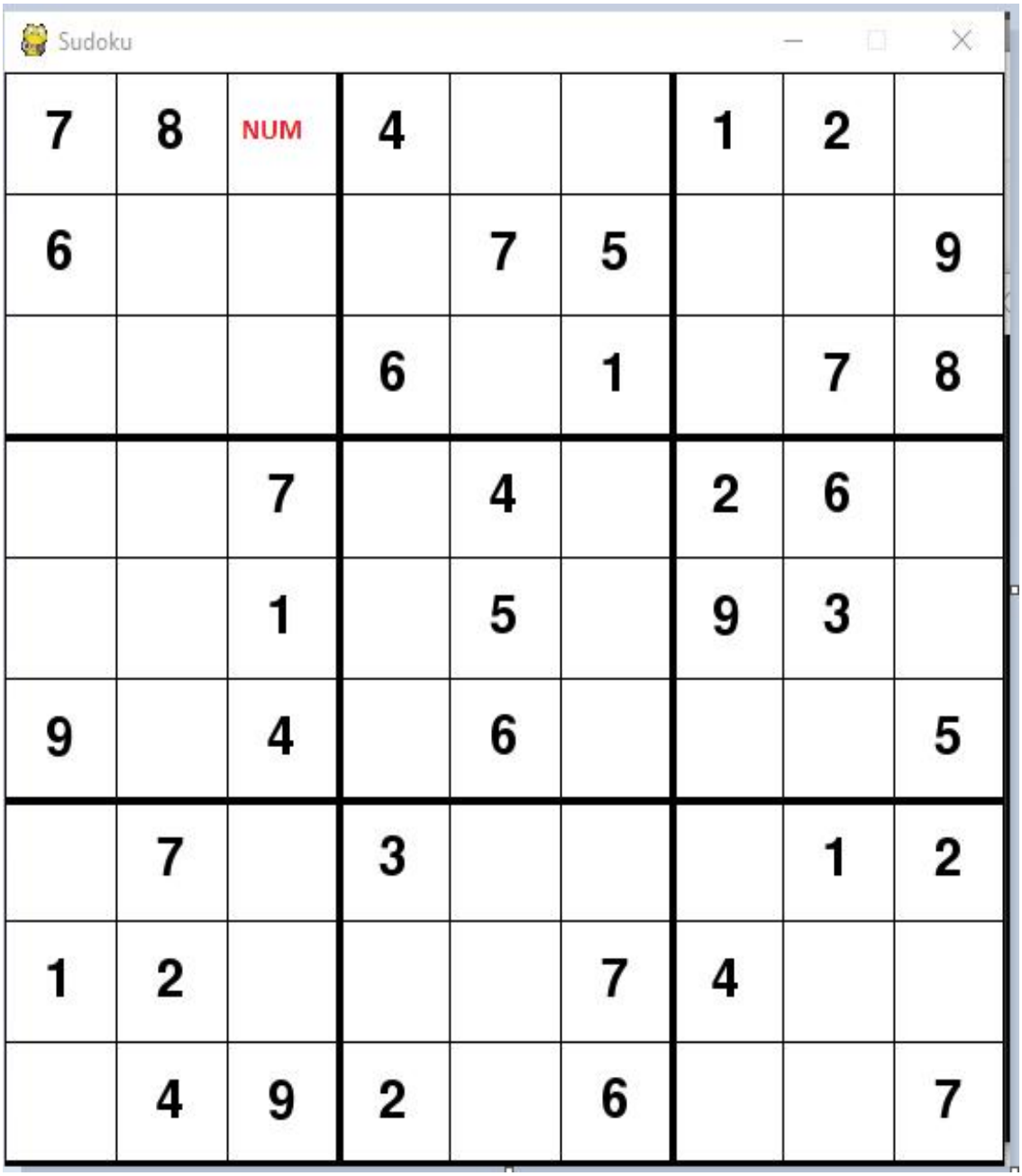
4.2.1. Biểu diễn sudoku

Ta biểu diễn bàn cờ vua bằng 1 ma trận vuông cấp9. int ip[9][9]

Quy ước :

- ip[pos[0]][i] == num and pos[1] != i : Kiểm tra số nhập vào theo hàng.
- ip[i][pos[1]] == num and pos[0] != i: Kiểm ra số nhập vào theo cột.
- ip[i][j] == num and (i, j) != pos : Kiểm ra số nhập vào theo ma trận 3x3.

4.2.2. Các khả năng lựa chọn cho số nhập vào



Mỗi **NUM** ở đây sẽ có 9 cách chọn tổng sẽ 9*9*9*…. .Vậy tối đa 1 lần giải sudoku mình sẽ có tối 54*9 trường hợp có thể xảy ra

4.2.3. Lựa chọn xi sao cho hiệu quả

Khi duyệt từng ô của ma trận mình sẽ so sánh sô vừa điền vào với cột, hàng và ô box của số đó(sẽ là số nhỏ nhất từ 1 đến 9) rồi duyệt theo chiều ngang

4.2.4. Cách ghi nhận trạng thái mới. Trả về trạng thái cũ.

Để ghi nhận bước đi hợp lệ i ta gán ip[row][col] = i

Để hủy 1 bước đi ta gán ip[row][col] = 0

4.2.5. Ghi nhận nhiệm vụ

Ma trận ip ghi nhận lại kết quả nghiệm . Nếu tồn tại h[x][y] = 0 ; thì không phải kết quả của bài toán . ngược lại h chứa đường đi cần tìm của nghiệm.

4.2.6. Mô tả

```
Backtracking(k) {
    for([Mỗi phương án chọn i(thuộc tập D)]) {
        if ([Chấp nhận i]) {
            [Chọn i cho X[k]];
            if ([Thành công]) {
                [Đưa ra kết quả];
            } else {
                Backtracking(k+1);
                [Bỏ chọn i cho X[k]];
            }
        }
    }
}
```

4.2.7. Đánh giá thuật toán

- **Ưu điểm :** Việc quay lui là thử tất cả các tổ hợp để tìm được một lời giải. Thế mạnh của phương pháp này là nhiều cài đặt tránh được việc phải thử nhiều trường hợp chưa hoàn chỉnh, nhờ đó giảm thời gian chạy.
- **Nhược điểm :** Trong trường hợp xấu nhất độ phức tạp của quay lui vẫn là cấp số mũ. Vì nó mắc phải các nhược điểm sau:

- Rơi vào tình trạng "thrashing": qúa trình tìm kiếm cứ gặp phải bế tắc với cùng một nguyên nhân.
- Thực hiện các công việc dư thừa: Mỗi lần chúng ta quay lui, chúng ta cần phải đánh giá lại lời giải trong khi đôi lúc điều đó không cần thiết.
- Không sớm phát hiện được các khả năng bị bế tắc trong tương lai. Quay lui chuẩn, không có cơ chế nhìn về tương lai để nhận biết đc nhánh tìm kiếm sẽ đi vào bế tắc.

4.2.8. Xây dựng các hàm:

- Hàm giải chương trình bằng phương pháp quay lui:

```
def solve(ip):
    find = find_empty(ip)
    if not find:
        return True
    else:
        row, col = find

    for i in range(1, 10):
        if valid(ip, i, (row, col)):
            ip[row][col] = i

            if solve(ip):
                return True

            ip[row][col] = 0

    return False
```

- Hàm kiểm tra:

```
def valid(ip, num, pos):

    # Check row
    for i in range(len(ip[0])):
        if ip[pos[0]][i] == num and pos[1] != i:
            return False

    # Check col
    for i in range(len(ip[0])):
        if ip[i][pos[1]] == num and pos[0] != i:
            return False

    # Check box
    box_x = pos[1] // 3
```

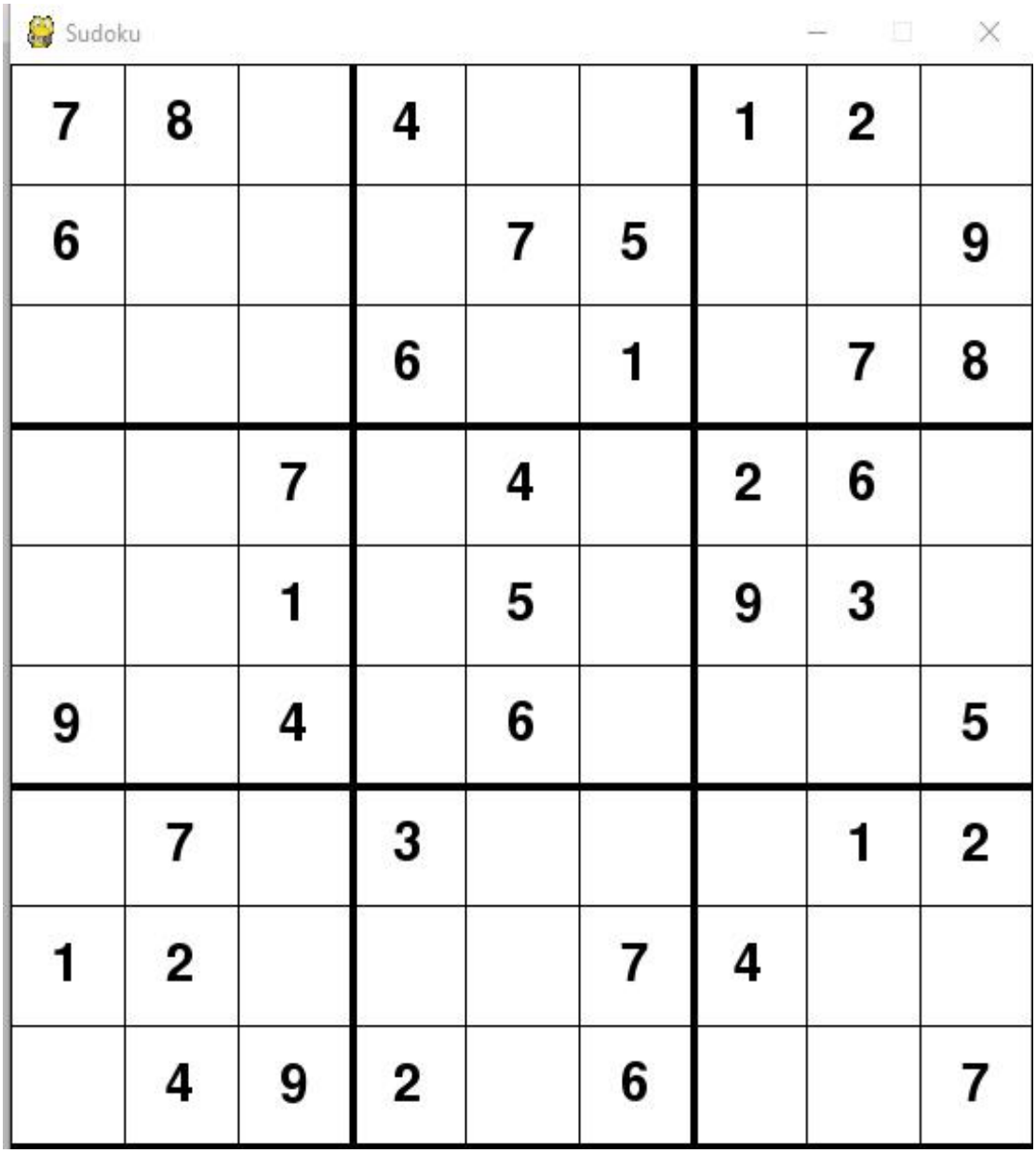
```
box_y = pos[0] // 3
for i in range(box_y * 3, box_y*3 + 3):
    for j in range(box_x * 3, box_x*3 + 3):
        if ip[i][j] == num and (i, j) != pos:
            return False

return True
```

4.3. Giới thiệu chương trình giải

4.3.1. Tổng quan:

Giao diện chương trình:



Màn hình này dùng để nhập vào các thông tin cơ bản là :

- Kích thước SUDOKU
- Các ô còn trống

Sudoku

7	8		4			1	2	
6				7	5			9
			6		1		7	8
		7		4		2	6	
		1		5		9	3	
9		4		6				5
	7		3				1	2
1	2				7	4		
	4	9	2		6			7

4.3.2. Cách thức chơi:

Đây là màn hình chính dùng chơi trò sudoku. Trên màn hình này ta có thể thực hiện các thao tác:

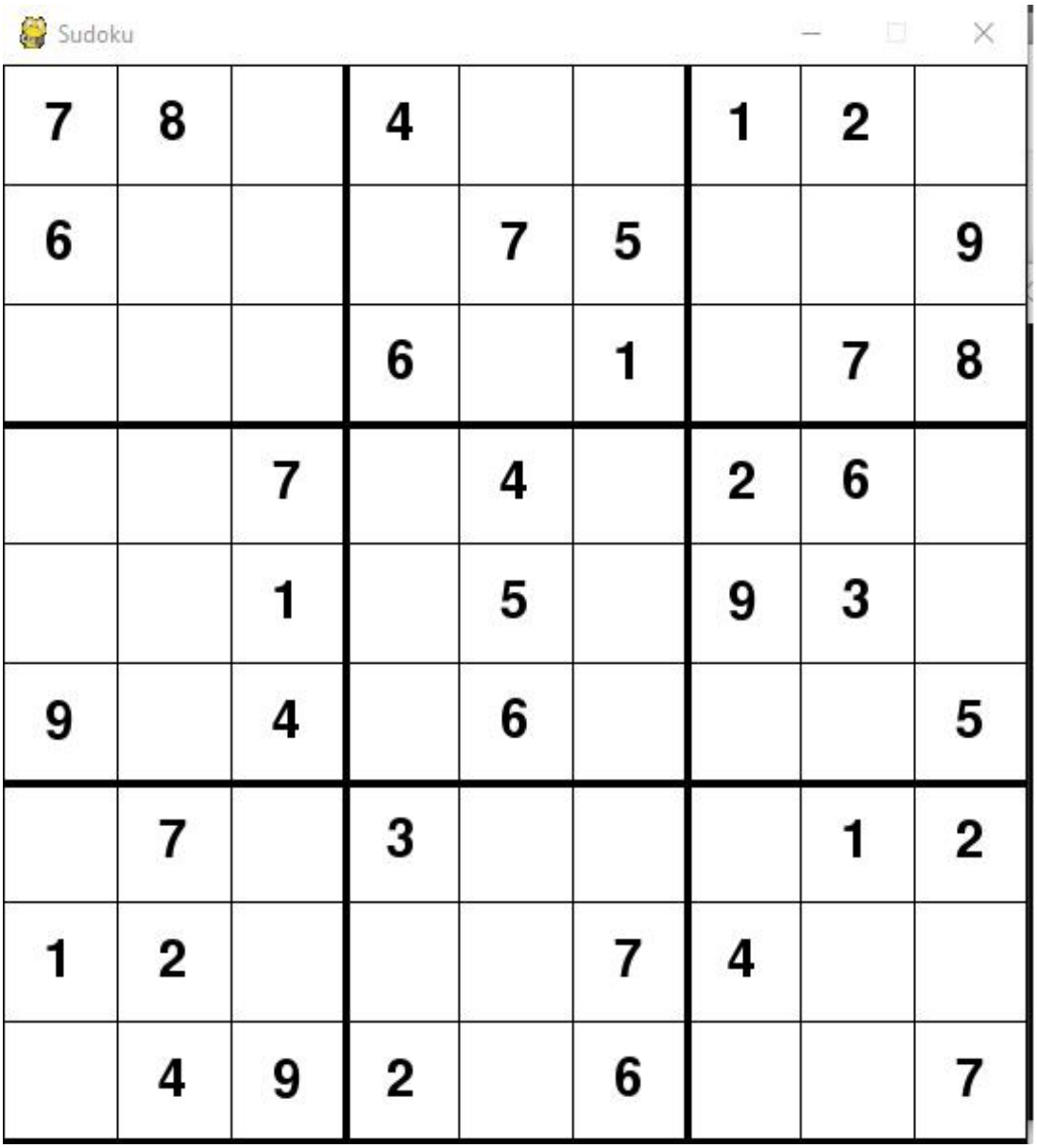
- Nhập 1 số mình nghĩ là đúng vào ô còn trống.
- Kiểm tra xem có số đó có đúng hay không bằng cách ấn phím enter.

7	8		4			1	2	
6				7	5			9
			6		1		7	8
		7		4		2	6	
		1		5		9	3	
9		4		6				5
	7		3				1	2
1	2				7	4		
	4	9	2		6			7

X Time: 24:3

CHƯƠNG 5: CÀI ĐẶT CHƯƠNG TRÌNH THỬ NGHIỆM

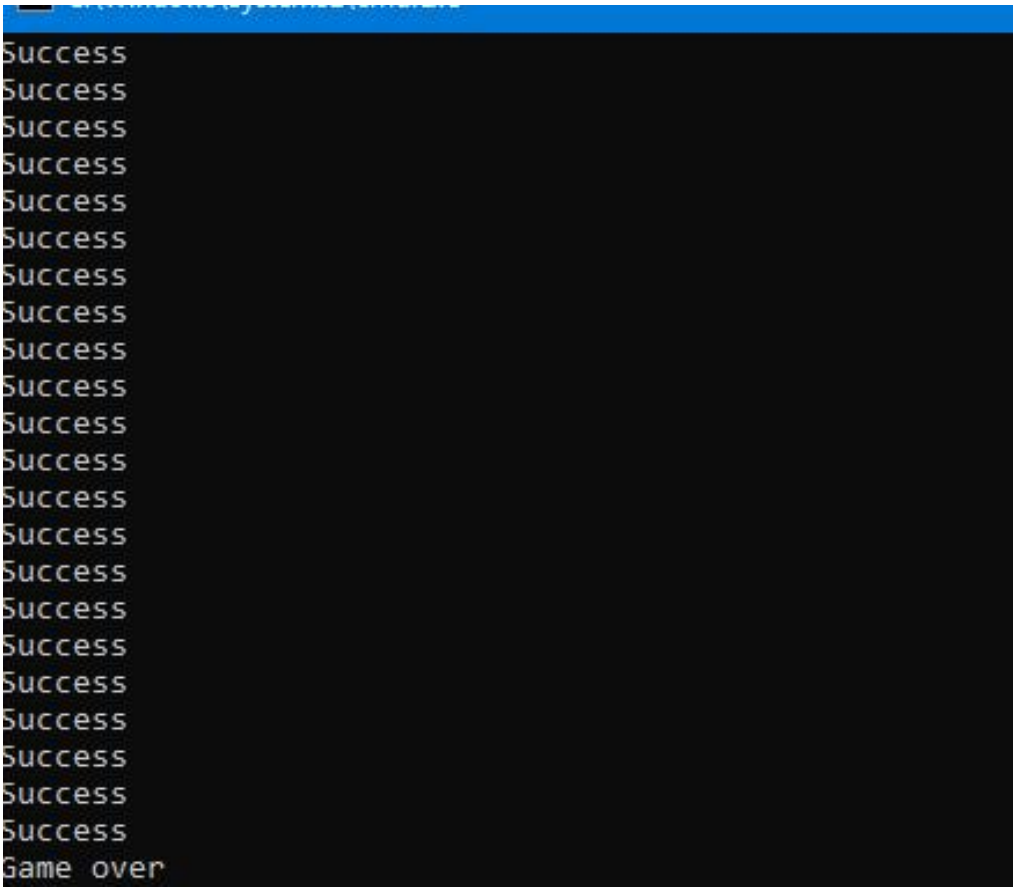
5.1. Chạy chương trình



5.2. Chạy chương trình kết thúc

7	8	5	4	3	9	1	2	6
6	1	2	8	7	5	3	4	9
4	9	3	6	2	1	5	7	8
8	5	7	9	4	3	2	6	1
2	6	1	7	5	8	9	3	4
9	3	4	1	6	2	7	8	5
5	7	8	3	9	4	6	1	2
1	2	6	5	8	7	4	9	3
3	4	9	2	1	6	8		7

Khi điền đủ các số thích hợp vào SUDOKU thì chương trình kết thúc ở CMD sẽ hiện lên game over:


Success
Success
Success

CHƯƠNG 6: ĐÁNH GIÁ VÀ HƯỚNG PHÁT TRIỂN

6.1. Những kết quả đạt được

Do thời gian thực hiện bài tiểu luận là có hạn và đề tài cũng tương đối phức tạp. Tuy nhiên, với sự nỗ lực của bản thân đồng thời nhờ sự hướng dẫn tận tình của thầy Nghiêm Văn Tính, và bạn bè nên đề tài đã hoàn thành đúng thời hạn quy định và thu được một số kết quả sau:

- Đã xây dựng thành công chương trình giải SUDOKU bằng thuật toán quay lui.
- Tìm hiểu khá kỹ về nội dung và kiến thức của thuật toán quay lui.
- Một kết quả quan trọng nữa là đã thu về cho bản thân những kinh nghiệm, kỹ năng lập trình, nghiên cứu tài liệu và nhiều kiến thức liên quan khác.

6.2. Những kết quả chưa đạt được

Bên cạnh những kết quả thu được đã nêu trên thì vẫn còn những kết quả chưa đạt được, đó là:

- Phần lý thuyết về kĩ thuật quay lui còn chưa nghiên cứu được nhiều và nghiên cứu sâu.
- Chương trình giải SUDOKU còn đơn giãn, chưa thật sự đẹp.
- Do thời gian thực hiện đề tài còn hạn chế và phần kiểm thử chương trình chưa được kỹ nên chương trình có thể còn có nhiều lỗi và chưa được tối ưu phần mã lệnh.

6.3. Hướng phát triển

Trong tương lai, đề tài có thể được phát triển và mở rộng theo một số hướng sau:

- Tối ưu thuật toán giải Sudoku bằng cách kết hợp thuật toán quay lui với các phương pháp khác như heuristic, branch and bound hoặc constraint propagation để giảm thời gian tìm kiếm lời giải.
- Mở rộng chương trình cho nhiều loại Sudoku khác nhau, chẳng hạn như Sudoku kích thước lớn hơn (16×16, 25×25) hoặc các biến thể Sudoku nâng cao.
- Cải thiện giao diện người dùng, giúp chương trình trực quan và thân thiện hơn, bổ sung các chức năng như gợi ý, kiểm tra lỗi và hiển thị từng bước giải.
- Phát triển chương trình thành một ứng dụng hoàn chỉnh, có thể chạy trên nhiều nền tảng như web hoặc thiết bị di động để người dùng dễ dàng sử dụng.

KẾT LUẬN

Trong quá trình thực hiện đề tài "Xây dựng phần mềm giải Sudoku bằng thuật toán quay lui", nhóm đã tiến hành nghiên cứu cơ sở lý thuyết về thuật toán quay lui, tìm hiểu đặc điểm của bài toán Sudoku và áp dụng thuật toán này để xây dựng chương trình giải Sudoku bằng ngôn ngữ lập trình Python.

Kết quả đạt được cho thấy thuật toán quay lui có thể giải quyết hiệu quả bài toán Sudoku bằng cách thử các khả năng có thể xảy ra và quay lui khi gặp trường hợp không thỏa mãn điều kiện. Chương trình đã thực hiện đúng chức năng đặt ra, có khả năng tìm ra lời giải hợp lệ cho bảng Sudoku 9×9 dựa trên các ràng buộc về hàng, cột và vùng 3×3.

Thông qua quá trình thực hiện đề tài, nhóm không chỉ củng cố được kiến thức về thuật toán quay lui, đệ quy và cấu trúc dữ liệu mà còn nâng cao kỹ năng lập trình, phân tích bài toán và xây dựng chương trình. Bên cạnh đó, việc kết hợp lý thuyết với thực hành đã giúp nhóm hiểu rõ hơn cách áp dụng các thuật toán vào giải quyết các bài toán thực tế trong lĩnh vực công nghệ thông tin.

Tuy nhiên, do thời gian thực hiện đề tài còn hạn chế nên chương trình vẫn còn một số hạn chế nhất định như giao diện chưa thật sự hoàn thiện, tốc độ xử lý trong một số trường hợp phức tạp chưa được tối ưu và chưa hỗ trợ nhiều biến thể Sudoku khác nhau. Trong tương lai, chương trình có thể được cải tiến bằng cách tối ưu thuật toán, nâng cấp giao diện và mở rộng chức năng để đáp ứng tốt hơn nhu cầu của người dùng.

Nhìn chung, đề tài đã hoàn thành được mục tiêu đề ra ban đầu và là một bước nghiên cứu có ý nghĩa trong việc tìm hiểu và ứng dụng các thuật toán giải bài toán logic trong lập trình.

TÀI LIỆU THAM KHẢO

1. Đinh Mạnh Tường, Cấu trúc dữ liệu và giải thuật, NXB khoa học kỹ thuật, 2001.

2. Python cơ bản - Võ Duy Tuấn.

3. Các giáo trình khác về ngôn ngữ lập trình Python.

* Một số Website:

- www.diendantinhoc.com
- www.thuvientinhoc.com
- www.slideshare.net
- http://congdongcviet.com/

4. Nghiên cứu phương pháp quay lui: https://www.slideshare.net/slideshow/nghin-cu-phng-php-quay-lui-v-ng-dng-gii-bi-ton-sudoku/240278281

5. https://luanvan.net.vn/luan-van/de-tai-giai-tro-sudoku-45852/